

Enhancing Finite State Machine Design Automation with Large Language Models and Prompt Engineering Techniques

Qun-Kai Lin*, Cheng Hsu*, and Tian-Sheuan Chang

Department of Electronics and Electrical Engineering, National Yang Ming Chiao Tung University, Hsinchu, Taiwan

*These authors contributed equally

Abstract—Large Language Models (LLMs) have attracted considerable attention in recent years due to their remarkable compatibility with Hardware Description Language (HDL) design. In this paper, we examine the performance of three major LLMs, Claude 3 Opus, ChatGPT-4, and ChatGPT-4o, in designing finite state machines (FSMs). By utilizing the instructional content provided by HDLBits, we evaluate the stability, limitations, and potential approaches for improving the success rates of these models. Furthermore, we explore the impact of using the prompt-refining method, To-do-Oriented Prompting (TOP) Patch, on the success rate of these LLM models in various FSM design scenarios. The results show that the systematic format prompt method and the novel prompt refinement method have the potential to be applied to other domains beyond HDL design automation, considering its possible integration with other prompt engineering techniques in the future.

I. INTRODUCTION

The emergence of Large Language Models (LLMs) has transformed various domains, with their potential applications expanding into new areas. One such field of interest is the automation of chip design, where LLMs have exhibited remarkable compatibility with Hardware Description Language (HDL) design. Mainstream LLMs, such as ChatGPT and Claude, have showcased their capability to aid in HDL design automation.

Prior research has investigated the use of LLMs for HDL design using natural language, identifying several effective prompt engineering (PE) techniques [1] [2]. Extending this groundwork, our study aims to compare the abilities of various LLMs and propose strategies for enhancing their performance. Drawing inspiration from recent evaluation frameworks commonly employed in the field [3] [4], we utilize a set of problems sourced from the Verilog learning platform, HDLBits, as our test problem set to assess the models' capabilities.

The primary objective of this study is to evaluate the performance of mainstream LLMs, specifically Claude 3 Opus, ChatGPT-4, and ChatGPT-4o, in designing finite state machines (FSMs). By leveraging the instructional content provided by HDLBits, we examine the stability, limitations, and potential approaches for improving the success rates of these models.

For this research, we meticulously selected 20 FSM design problems from HDLBits. We employ a systematic format to

address the problem description. By endeavoring to solve these problems using the chosen LLMs in conjunction with the systematic format prompt, we record their success rates and analyze the underlying causes of any encountered failures. Our goal is to develop and propose strategies that can boost the performance of these models in HDL design tasks.

Another key focus of this paper is to explore the impact of using the prompt-refining method, To-do-Oriented Prompting (TOP) Patch. This is an efficient approach to refine the systematic format prompt and achieve better performance. By examining the impact of the TOP Patch on the success rates of LLMs in various FSM design scenarios, we gain insights into the effectiveness of this approach. Furthermore, we explore the potential applications of the TOP Patch beyond FSM design, considering its possible integration with other prompt engineering techniques in the future.

Through the comparative analysis, the systematic format prompt, and the novel prompt refinement method, we aim to contribute to the ongoing advancement of LLMs in the field of HDL design automation. By identifying the strengths and weaknesses of each model, we strive to pave the way for more effective and efficient design processes. Ultimately, our goal is to further enhance the capabilities of LLMs in the realm of automatic chip design, enabling them to become invaluable tools for engineers and researchers in the field.

II. EFFECTIVE PROMPT ENGINEERING TECHNIQUES

To enhance the LLMs' comprehension of the provided prompt, we employ a systematic approach using the markdown format. This method allows for clear distinctions between each part of the prompt, such as the specification, I/O list, and module functionality.

By utilizing itemized lists to explain several points, the emphasis becomes more apparent compared to using a long paragraph. Additionally, the convenience of table construction enables the systematic presentation of I/O details, the conversion of visualized waveform into readable lists with signal names and values, and the construction of state transition tables. By leveraging the markdown format, we aim to improve the LLMs' understanding of the given prompt.

Act as a professional SystemVerilog programmer. You are going to design a module based on the specifications.

Specification

The game Lemmings involves critters with fairly simple brains. So simple that we are going to model it using a finite state machine.

... (remaining problem description)

Example Behavior (truncated)

clk	1	2	3	4	5	
bump_left	0	0	0	0	0	...
bump_right	0	0	1	0	0	
walk_left	1	1	1	1	1	
walk_right	0	0	0	0	0	

Module Declaration

```

““systemverilog
module top_module(
input clk,
input areset, // Freshly brainwashed Lemmings walk left.
input bump_left,
input bump_right,
output logic walk_left,
output logic walk_right
);
...
endmodule
““

```

PROMPT I: Systematic markdown form of Lemmings1

Focusing on the FSM design problems selected from HDLBits, the design of the state transition table undoubtedly represents the high-level language description of the FSM design. We preserve the problem description excerpted from the HDLBits website and transform the example waveform and state description graph into markdown table form. The prompt begins with the command “Act as a professional SystemVerilog programmer.” We then place the problem description in the “Specification” section, followed by the “Example waveform” and “Module declaration” sections, where we include the module name and I/O ports. The final part of the prompt asks the LLMs to implement the FSM in SystemVerilog. See an example in [Prompt I](#).

III. EVALUATING THE PERFORMANCE OF LLMs IN SYSTEMVERILOG FSM DESIGN

A. Experimental Setup and Methodology

For the 20 FSM design problems selected from HDLBits, we evaluate each problem five times (using five independent chat sessions) through Claude 3 Opus, ChatGPT-4, and ChatGPT-4o, requiring them to generate SystemVerilog code.

The problems are nearly identical to those on the HDLBits website, with minor modifications to better suit the markdown

format. No additional hints about the problems are provided to the LLMs. For problems that require information from the previous problem, we additionally supply the correct version of the transition table or SystemVerilog code generated by the LLM itself in the preceding problem. Furthermore, to test their SystemVerilog writing capabilities, only the module interface is given. This test aims to assess the basic syntax knowledge and FSM design ability of the LLMs.

B. Comparative Analysis of LLM Performance

Problem	Claude 3 Opus	ChatGPT-4	ChatGPT-4o
Fsm1	5	5	4
Fsm1s	5	4	5
Fsm2	5	3	4
Fsm2s	4	5	2
Fsm3comb	5	5	5
Fsm3onehot	0	0	0
Fsm3	5	4	4
Fsm3s	5	3	1
Exams/ece241 2013 q4	0	0	0
Lemmings1	1	0	1
Lemmings2	1	0	2
Lemmings3	4	3	3
Lemmings4	0	0	0
Fsm onehot	0	0	0
Fsm ps2	3	0	0
Fsm ps2data	0	0	0
Fsm serial	0	0	0
Fsm serialdata	0	0	0
Fsm serialdp	0	0	0
Fsm hdlc	0	0	0
# of Problem Solved Successfully	11	8	10
Total Success Rate	41%	32%	31%

TABLE I: Assessing the Single-Shot Capabilities of LLMs in Automating FSM Design: Results from 5 Independent Trials

The results are presented in [Table I](#), showcasing the strengths and weaknesses of these models when confronted with various FSM design problems. The common syntax errors include multi-driver issues in multiple always blocks, missing or improperly positioned logic declarations, and confusion between `begin...end` and brackets.

One type of problem that poses a challenge is the one-hot design, which requires deriving equations through inspection, i.e., directly forming Boolean equations. All three models perform poorly in this regard. Across all questions, they generally struggle with handling truth tables or Karnaugh maps. While they correctly generate most logic equations, they often make a few mistakes.

For more intricate designs with lengthy descriptions, the models often misinterpret the state-transition logic relationship. Furthermore, the number of states influences the likelihood of FSM correctness. If fewer states are used to form the FSM, individual states might contain unusual and convoluted logic, which is a primary factor leading to failures.

Claude 3 Opus achieves the highest success rate among the three models. It demonstrates stability in solving problems within its capabilities and exhibits no syntax error.

To-do

1. Explain synchronous reset and give a basic example.
2. Tell the difference between synchronous and asynchronous reset design in SystemVerilog implementation.
3. Implement the above design specifications in SystemVerilog.

PROMPT II: TOP Patch for Synchronous Reset FSM design

ChatGPT-4 effectively comprehends and processes the provided information. It generally performs well but occasionally makes mistakes that deviate from the intended solution. Its stability is relatively less consistent compared to the other models.

ChatGPT-4o performs similarly to *ChatGPT-4* and even surpasses it in terms of correctness and stability. However, it is noteworthy that it defaults to treating designs as having asynchronous reset. When dealing with designs that require synchronous reset (such as Fsm2s & Fsm3s in Table I), it often fails to adhere to the prompt. To cope with this defect, we utilize a special refinement strategy that will be further mentioned in Section IV.

IV. TOP PATCH: OVERCOMING LLM LIMITATIONS

Even with systematic markdown format prompt engineering, more complex designs may require more sophisticated design descriptions, which can hinder the effectiveness of the received feedback. To address this issue, we need to provide additional hints on important signals or pivotal concepts to help LLMs focus on the key points and clarify essential concepts. Here, we introduce an effective strategy to improve the performance of LLM register transfer level (RTL) code generation: To-do-Oriented Prompting (TOP) Patch.

TOP Patch is an efficient method to solve problems resulting from complex specifications or concepts. It is an extra snippet that can be appended to the end of the systematic markdown format prompt, boosting the quality of LLMs' responses and consequently increasing the success rate of complex RTL design.

TOP Patch introduces a "To-do" section that is added to the end of the prompt, assisting LLMs in concentrating on specific points. The contents of this section are primarily composed of essential signal specifications, constraints, or important concepts, generated either by human experts or LLMs themselves, and presented point by point. LLMs tend to execute these points in order, making the sequence of the contents a crucial factor. Examples of a TOP Patch utilized in solving particular problems can be found in Prompt II and Prompt III.

By utilizing the TOP Patch strategy, we can address the limitations of certain LLMs when dealing with unique or challenging scenarios. The approach involves minimal human intervention, making it an efficient and practical solution. The inclusion of a TOP Patch has been shown to significantly

To-do

1. Explain "derive equations by inspection".
2. List out every situation that will result in each next_state.
3. Implement the entire SystemVerilog module for the state machine using the above two results.

PROMPT III: TOP Patch for One-hot FSM design

improve the success rate of LLMs in handling these special cases.

V. EXPERIMENTAL RESULTS

A. Addressing Synchronous Reset Issues in ChatGPT-4o with TOP Patch

Through observation, we have discovered that *ChatGPT-4o* struggles to solve the synchronous reset FSM design problems in Table I, specifically Fsm2s and Fsm3s.

The primary issue is that *ChatGPT-4o* consistently utilizes asynchronous flip-flops to implement the FSM when prompted solely with the plain problem description.

In Prompt III, the first two points serve to ensure that the LLM understands the distinction between synchronous and asynchronous reset and possesses the knowledge to correctly implement both types of flip-flops.

By incorporating this additional verification step, the TOP Patch successfully increases the success rate from 30% to 70%, as demonstrated in Table II. This result showcases the capability of the TOP Patch in assisting LLMs to overcome specific design limitations.

Problem	ChatGPT-4o	ChatGPT-4o + TOP Patch
Fsm2s	2	4
Fsm3s	1	3
Success Rate	30%	70%

TABLE II: Single-Shot Capabilities of ChatGPT-4o utilizing TOP Patch in Synchronous Reset FSM Design

B. Improving One-hot FSM Design Success Rates using TOP Patch

For the one-hot FSM design problems in Table I, namely Fsm3onehot and Fsm onehot, the design requirements involve deriving equations through inspection, which restricts the design method.

If prompted solely by the plain problem description, LLMs tend to either ignore or struggle with the complex request. Consequently, specific hints are provided using the TOP Patch approach.

In Prompt II, the first two points aim to draw the LLMs' attention to the pivotal elements that significantly affect correctness. The first point ensures that LLMs fully comprehend and understand the special requirements of this problem.

To-do

1. Read the example behavior first carefully. Elaborate the reasoning behind these behaviors.
2. design a FSM for the above specification. Give the state transition table with the outputs details that contains every input condition for each state.
3. implement the entire SystemVerilog module for the state machine.

PROMPT IV: Multi-Shot using TOP Patch & CoT for Lemmings1

The second point further requests LLMs to summarize every transition situation. By providing these hints, we ensure an increase in the accuracy and success rate of these unique one-hot FSM designs.

After applying the TOP Patch, a noticeable improvement in the success rate is observed. As shown in Table III, the success rates of all three LLMs demonstrate the ability for them to solve the one-hot FSM design problems. Impressively, Claude 3 Opus demonstrates the best performance, achieving a 90% success rate when using the TOP Patch.

Problem	Claude 3 Opus	ChatGPT-4	ChatGPT-4o
Fsm3onehot	4	2	1
Fsm onehot	5	1	3
Success Rate	90%	30%	40%

TABLE III: Single-Shot Capabilities of LLMs utilizing TOP Patch in One-hot FSM Design

C. Tackling Complex FSM Designs with TOP Patch and Chain-of-Thought Prompting

For much more complex problems, it seems impossible to complete them in a single attempt. It may require feedback to revise the error points or inspiration from manual hints.

In this case, the TOP Patch strategy can also work as a medium to emphasize the tasks to be executed in the next chat section. It provides a workflow and can act as a framework integrated with Chain-of-Thought (CoT), one of the famous PE techniques. Prompt IV shows an example for Lemmings1 in HDLBits that uses TOP Patch to provide a CoT flow.

Furthermore, in cases where a single task may contain a large amount of response, it is acceptable and potentially beneficial to break it into several prompts. This approach ensures and checks the quality of the current stage, allowing the remaining stages to be gradually completed.

After the initial SystemVerilog codes are generated, some advanced feedback methods can be introduced [5]. Based on the severity of the error, error feedback (directly copying the compilation info), human feedback (manually checking the problems that occurred in the waveform), and advanced feedback (engaging in conversation with LLMs to obtain advice) take place in the secondary debugging step.

VI. CONCLUSION AND FUTURE DIRECTIONS

This research provides a comparison between three major LLMs in terms of their HDL generation quality. Utilizing a problem set from HDLBits, we employ a systematic and effective prompt method and propose an efficient prompt refinement approach called TOP Patch. This refinement method yields impressive results in performance enhancement.

Moreover, the generation of TOP Patch can be accomplished by LLMs that require further training and fine-tuning. In the future, once TOP Patch automation becomes stable, it has the potential to be integrated into the feedback system of automated HDL generator frameworks [4]. By incorporating this prompt refinement solution, we can achieve a higher level of chip design automation.

Looking ahead, the automation of TOP Patch generation through fine-tuned LLMs opens up exciting possibilities for enhancing the efficiency and accuracy of HDL design automation frameworks. By seamlessly integrating TOP Patch into the feedback loop of these frameworks, we can create a more robust and adaptive system that continuously refines prompts based on the specific design context and requirements.

Furthermore, the principles and techniques introduced in this research have the potential to be applied to other domains beyond HDL design automation. The systematic prompt method and the concept of prompt refinement through TOP Patch can be explored in various fields where LLMs are utilized for code generation, natural language processing, or creative writing tasks.

REFERENCES

- [1] Y. Lu *et al.*, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.
- [2] B. T. Hammond Pearce *et al.*, "Dave: Deriving automatically verilog from english. in 2020 acm/ieee 2nd workshop on machine learning for cad (mlcad). 27–32," 2020.
- [3] M. Liu *et al.*, "Verilogval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.
- [4] S. Thakur *et al.*, "Autochip: Automating hdl generation using llm feedback," *arXiv preprint arXiv:2311.04887*, 2023.
- [5] J. Blocklove *et al.*, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.